

Project Blueprint: Pupils Management System

1. Executive Summary

This document outlines the requirements for developing "Pupils Management", a comprehensive school management system inspired by the functionalities of IBEAMS. The system is designed to manage all aspects of a school's administrative and academic operations, including student enrollment, class management, attendance, fees, examinations, reporting, and parent/teacher communication.

THE SYSTEM OF:

- The System
- The Teacher's Portal
- The Parent Portal
- The Android Daily Register App

Simplified Management

Our purpose built features ensures extreme ease of use that requires minimum expertise and skills.

Full Control

Every aspect of management is covered with regular updates, improvements and feature requests.

Responsive Design

Elegantly designed to fulfil a seamless graphical user interface for the best user experience.

List of Features, Students, Teachers, Admin staff

Teacher's API (for own web site), Student API (for own web site) to update student info, Messaging system between teachers and administration, Help section

Manage Holidays Opening Hours Teacher Log System Student monthly register, Student daily register, Android tablet app for daily register entry, Incident reporting, Downloads & Tools, Fees management, Fees reports, Fees Receipt Sending, Fees reports, Full student info, Quick data access, Newsletter feature, End of term PDF/Email report

For Parents

Homework Boost

your child's education and learning with the homework feature that allows you to provide feedback and mark as complete. No more searching through emails, just login and view by date.

Attendance

View attendance of each child, monthly reports with a tally of absent, late, authorised and unauthorised.

List of Features Update your details Add emergency contacts Enable/disable contacts to allow access to the portal Contact the management View homework for all children View all holidays View and download end of term reports View and download end of year reports View incidents involving your child View exam results View test results Check fees history and payment records View child attendance View merits Download certificates and awards End of term PDF/Email reports

The application will be developed for three primary platforms:

- Windows Application: For administrative staff and school management.
- Android Application: For teachers to take registers, view schedules, and communicate on the go with parents and students .

.iOS application For teachers to take registers, view schedules, and communicate on the go with parents and students.

A robust, relational database will serve as the central data repository.

2. Core Modules & Features

The system will be organized into the following core modules:

- Administration/Setup: Academic Year, School Profile, User Management, System Settings.
- Student Management: Add/Edit/Import Students, Student Status, Siblings, Contacts (Parents/Guardians).
- Academic Management: Class Management, Teacher Assignment, Timetable Scheduling.
- Attendance: Daily & Monthly Registers, Reporting, SMS Notifications for Absences.
- Examinations: Official Exams & Progressive Tests, Results Entry, Grading.
- Reporting: End of Term/Year Reports (Performance, Behavior, Attendance), Certificates.
- Communication: Teacher & Parent Portals, SMS Integration, Email Notifications.
- Student Diaries: Homework, Syllabus Tracking, Hifz/Revision Logs.
- Merits & Store: Points System, Achievements, Product Store with Merits/Currency and certificates generating system for end of semester.
- Fees & Finance: Fees Structure Creation, Invoice Generation, Payment Tracking.

3. Database Schema & Management

The database will be a relational structure, likely using SQL (e.g., PostgreSQL, MySQL, or SQLite for mobile). The following are the key entities and their relationships.

3.1 Key Entities (Tables)

- academic_years
 - id (PK), title, start_date, end_date, is_current (BOOLEAN)
- users (Staff/Administrators/Teachers)
 - id (PK), email (UNIQUE), password_hash, first_name, last_name, role (admin, teacher), status (active, inactive), permissions (JSON)
- teachers
 - id (PK), user_id (FK), is_form_tutor (BOOLEAN), other_details
- students
 - id (PK), first_name, last_name, date_of_birth, status (waiting_list, accepted, current, left, suspended), enrollment_date
- contacts
 - id (PK), first_name, last_name, email (UNIQUE), phone, address, password_hash
- student_contacts
 - student_id (FK), contact_id (FK)
- classes
 - id (PK), name, age_group_id (FK), form_teacher_id (FK to teachers), academic_year_id (FK), start_date, end_date
 - class_teachers (For assigning multiple teachers)
 - id (PK), class_id (FK), teacher_id (FK), permissions (JSON)
- timetable_slots
 - id (PK), class_id (FK), day_of_week (INT), start_time, end_time, teacher_id (FK), location, is_register_taken (BOOLEAN)
- class_history
 - student_id (FK), class_id (FK), start_date, end_date

- daily_registers
 - id (PK), student_id (FK), class_id (FK), date (DATE), status (present, absent, late), minutes_late, is_authorised (BOOLEAN)
- monthly_registers
 - id (PK), student_id (FK), class_id (FK), year_month (DATE), absent_count, late_count
- exams
 - id (PK), name, type (official, progressive), date, status (open, closed)
- exam_tests
 - id (PK), exam_id (FK), subject, class_id (FK), max_marks, teacher_id (FK)
- exam_results
 - id (PK), exam_test_id (FK), student_id (FK), marks_obtained
- end_of_term_reports
 - id (PK), title, term_ending_date, academic_year_id (FK), status (active, ready)
- report_entries
 - id (PK), report_id (FK), student_id (FK), class_id (FK), behavior_score (INT 1-5), effort_score (INT 1-5), progress_score (INT 1-5), teacher_comment (TEXT)
- events
 - id (PK), name, date, duration, opt_in_deadline, price, description, is_active (BOOLEAN)
- event_registrations
 - event_id (FK), student_id (FK), timeslot, payment_status (paid, pending)
- products
 - id (PK), name, price, merit_price, stock
- orders
 - id (PK), student_id (FK), product_id (FK), order_date, status (ordered, paid, delivered)
- merits
 - id (PK), student_id (FK), teacher_id (FK), merit_type_id (FK), points, date_awarded
- merit_types
 - id (PK), name, points_value, is_achievement (BOOLEAN)
- sms_logs
 - id (PK), recipient_number, message, status (sent, failed), sent_via (app, provider)

fees_structures

- id (PK), academic_year_id (FK), class_id (FK), fee_item (e.g., Tuition), amount, frequency (termly, monthly)
- invoices
 - id (PK), student_id (FK), fees_structure_id (FK), generation_date, due_date, total_amount, status (unpaid, paid, partially_paid)
- payments
 - id (PK), invoice_id (FK), amount, payment_date, method

3.2 Database Management Information

- Database Server: For the Windows application, a central SQL server (e.g., PostgreSQL, MySQL) is required. The mobile app will act as a client to this server.
- API: A secure RESTful API will act as the intermediary between the Windows/Android applications and the database. All data interactions will be through API calls.
- Authentication: JWT (JSON Web Tokens) will be used for secure user authentication and authorization.
- Migrations: Database migrations (e.g., using Alembic for Python, Laravel's Migrations, or Flyway) should be used to version-control the database schema and make updates seamless.
- Offline Capability: For Android, implement a local database (e.g., SQLite/Room) that can synchronize with the central server when an internet connection is available. This is crucial for teachers taking registers.

4. Technical Architecture & Stack Recommendations

4.1 General

- Architecture: Client-Server (RESTful API). Multiple clients (Windows Desktop App, Android App, Future Web Portal) communicate with a centralized API server.

- API Framework: Node.js (with Express.js), Python (with Django REST Framework or FastAPI), or C# (.NET Core). Choose based on the team's expertise.
- Database: PostgreSQL (robust, feature-rich) or MySQL.

4.2 Windows Application

- Framework: .NET MAUI (Multi-platform App UI) or WPF (Windows Presentation Foundation). MAUI allows for a single codebase to target Windows, Android, and iOS, but for pure Windows WPF is a very mature and stable choice.
- UI Design: XAML for a clean, responsive, and data-driven interface.

4.3 Android Application

- Native: Kotlin with Android SDK. Provides the best performance and access to device features (SMS, Camera).
- Cross-Platform: Flutter (Dart) or React Native (JavaScript). Both allow for a single codebase for Android and iOS (if needed). Flutter is recommended for its performance and UI consistency.
- Offline Storage: Room Persistence Library (Kotlin) or Hive (Flutter).

4.4 Hosting & Infrastructure

- Server: Cloud-based (AWS, Google Cloud, Azure) or an on-premise server.
- Web Server: Nginx or Apache.
- SSL/TLS: Essential for encrypting data in transit and securing the API.

5. Detailed Module Implementation Notes

5.1 Setup & Configuration

- Academic Year: The system must be built around the concept of an academic year. All core entities (students, classes, fees) are linked to a year. A "New Academic Year Wizard" is essential to migrate data (classes, students, fee structures) without losing historical records.
- School Profile: Easily customizable settings (logo, public address, financial/sender emails) that are used globally in PDFs and emails.

5.2 People Management (Students, Teachers, Contacts)

- Students & Contacts (Parents): Implement a many-to-many relationship. A contact can be associated with multiple students (siblings). Contact emails must be unique for parent portal login.
- Import/Export: Robust CSV import and export for all entities.
- Student Status: The student lifecycle (waiting_list -> accepted -> current -> left) must be managed. Status changes should trigger actions (e.g., "Student Left" wizard to delete invoices/register entries).

5.3 Class Management & Timetables

- Classes & Teachers: A class has one "Form Teacher" but can have many additional teachers with granular permissions (register, reports, homework, etc.).
- Timetable Templates: Allow creating reusable timetable templates (e.g., Morning, Evening, Ramadhan) and applying them to multiple classes.
- Classroom Management: A separate feature to add and manage classrooms/buildings and assign them to timetable slots.

5.4 Attendance (Registers)

- Daily vs. Monthly: Design the register system to support both. Daily is for day-to-day tracking; Monthly is a summary for reporting.
- Data Flow: Daily register data should be able to auto-fill monthly registers.
- Teacher Experience: The teacher's workflow should be streamlined. They should see their assigned timetable for the day. By default, all students are marked "Present." The teacher toggles to "Absent" or "Late."
- Offline First: The Android app must function fully offline. All register data is saved locally and synced when a connection is available.

5.5 Fees Management

- Fee Structure: Define fee items (Tuition, Transportation) per class or per student. Invoices are generated based on this structure for all students.
- Invoicing & Payments: Generate invoices, track payments, and handle partial payments. Creating separate invoices for class-specific fees (Independent Fees) is a required feature.

5.6 Examinations & Reporting

- Exams vs. Progressive Tests: Both have the same structure but serve different purposes. Official exam results are used in formal end-of-term/year reports; progressive test results are for ongoing tracking.
- End of Term/Year Report: A sophisticated system that pulls data from registers (attendance), exams (results), and manual teacher input (behavior, effort, progress). The manual scores (1-5) are converted to descriptive sentences via configurable "Paragraph Templates."

5.7 Communication & Portals

- Portals (Teacher/Parent/Student): The system does not have a built-in public portal. Instead, it provides "Portal Files" (PHP files) that an institution can host on their own website. These files act as a proxy, securely communicating with the API. This is a unique design choice that prioritizes security and branding for the school.
- SMS Integration: Supports two methods:
 1. Via Android App: Uses the device's SIM card to send SMS. This is a cost-effective method but requires the app to be running on a physical device with an active SIM.
 2. Via Providers: Integrate with SMS gateway APIs like ClickSend or VoodooSMS.

6. Key Features & Implementation Considerations

- Security: The system URL should be kept private. Portal files are used to keep this URL hidden and ensure secure communication.
- Data Migration:
 - Provide tools for each step of the academic year setup (migrate classes, students, fee structures).
 - Allow importing data via CSV.
- Customization:
 - Custom Fields for students and other entities.
 - Customizable Date/Time formats.
 - Age Group customization.

- Scalability: The application should be designed to handle a large volume of users, data, and concurrent requests. Use caching, database indexing, and efficient queries.
- Notifications: Email notifications for status changes, fee reminders, and new reports. SMS notifications for absences.
- Reporting: Generate dynamic PDF reports (Student Reports, Class Lists, Financial Reports) using a library like iText, PDFBox, or wkhtmltopdf.

7. Project Timeline (Rough Estimate)

- Phase 1: Setup & Core Data (Months 1-2):
 - Database design and migration system.
 - API development (Authentication, Users, Students, Teachers, Classes).
 - Windows Application UI (Login, Dashboard, School Setup).
- Phase 2: Academics & Attendance (Months 3-4):
 - Implement Timetable & Register logic.
 - Android App (Login, Teacher Dashboard, Offline Register Sync).
 - Develop SMS integration logic.
- Phase 3: Fees, Examinations & Reporting (Months 5-7):
 - Fees structure and invoice generation logic.
 - Exam system and results entry.
 - End of Term Report generation logic.
- Phase 4: Communication, Portals & Finalization (Months 8-9):
 - Complete Portal File specifications and deployment guide.
 - Merits, Store, and Student Diary modules.
 - Testing, bug fixing, and deployment.

8. Glossary

- PK: Primary Key (Unique identifier for a row in a database table).
- FK: Foreign Key (A field in one table that refers to the Primary Key of another table).
- RESTful API: An application programming interface that uses HTTP requests to GET, POST, PUT, and DELETE data.
- JWT: JSON Web Token, a compact and self-contained way for securely transmitting information between parties as a JSON object. Used for authentication.
- CSV: Comma-Separated Values, a simple text format for storing tabular data.

- cPanel: A popular web-based control panel for managing web hosting.
- FTP: File Transfer Protocol, a standard network protocol used to transfer files.
- SSL: Secure Sockets Layer, a standard security technology for establishing an encrypted link between a web server and a browser.